Table 3.5 Key features of TypeScript's type system

Topic	Fundamentals	Applied skills	Advanced
Type annotations	Use basic types: `string`, `number`, `boolean`, `any`.	Annotate arrays, objects, and function signatures.	Design complex types with generics, conditional types, and mapped types for scalable APIs.
Type inference	Explain how TypeScript infers types.	Leverage inference for clean code; identify when explicit annotations improve clarity.	Debug inference pitfalls, architect type-safe APIs balancing inference and explicit types.
Structural typing	Understand duck typing and structural compatibility.	Compare interface vs type; use compatibility checks to refactor code safely.	Solve complex type compatibility issues; design extensible systems that leverage structural typing.
Union & intersection types	Use union (`|`) and intersection (`&`) types.	Model flexible APIs and handle discriminated unions.	Architect type-safe patterns with advanced unions and intersections.
Literal & enum types	Use string/number literals, define enums.	Model precise APIs, use `const enums` for optimization.	Replace enums with union literals for safety; optimize enums for tree-shaking in bundles.

With a grasp of TypeScript's type system, we can now explore common interview questions to see these concepts in action.